

The RL v2 Pipeline

Learning transfer functions from instructions,
measured by what the render actually shows

Kevin Kunkel · Master’s thesis · Working document, 16 September 2026

Note: This document describes the pipeline as built and measured, including the two formulations that failed. Every number below comes from a run recorded in `out/`; the commands that reproduce them are in Section 12.

1. What the pipeline has to answer

A transfer function (TF) maps voxel intensity to colour and opacity. It decides what a volume rendering shows: whether the ribs are visible, whether the skin hides them, whether the lungs read as empty space or as tissue. Designing one by hand is fiddly, and the thesis asks whether a policy can be **learned** that does it from an instruction — ultimately spoken, ultimately in VR.

Three questions sit underneath that, and the pipeline separates them deliberately:

Q1 — Can “what the instruction asks for” be measured at all? Without a measurement grounded in the rendered image, nothing downstream means anything.

Q2 — Given such a measure, can a policy learn to satisfy instructions on patients it has never seen?

Q3 — Can the measure come from a human instead of a formula? This is the thesis’s core argument, and the part the collected preference data serves.

The pipeline answers Q1 and Q2 and builds the apparatus for Q3.

2. Why the previous pipeline could not answer them

The earlier system optimised `mass_fraction`: the integral of the TF’s opacity curve over a tissue’s Hounsfield band. Measured on `ct_chest`:

Change to the transfer function	Bone visible in the render	<code>mass_fraction(bone)</code>
default	0.3 %	0.26
bone peak height $\rightarrow$ max ¹	0.3 %	0.43
fat and soft-tissue peaks $\rightarrow$ 0	4.5 %	0.26

Table 1: The old objective rewarded a change that altered nothing on screen, and ignored the change that made bone nine times more visible.

`mass_fraction` never reads the volume. It returns the same value for a chest CT and a synthetic phantom. Three structural problems followed from it: the reward did not measure the image, the effective strategy (clearing whatever occludes the target) was outside a one-dimensional action space, and the policy observed nothing about the patient.

3. Measuring what a render shows

3.1. The estimate

Rendering inside an RL loop is too slow (50–210 ms) and a rendered image cannot say which tissue produced which pixel. So the pipeline estimates both cheaply.

¹The only action the previous agent had.

Per volume, once: resample into one cube per view (side 80, isotropic), one cube per each of six fixed cameras, so that the cube’s first axis runs front to back; label every sample by anatomy; quantise intensities to a 256-entry table. Per call: look up opacity and colour, correct opacity for step length, composite front to back, and accumulate per class.

$$\text{vis}_c = \frac{1}{N_{\text{rays}} \sum_{\text{samples} \in c} T \cdot \alpha}, \quad \text{bright}_c = \frac{\sum_{s \in c} T \alpha \cdot \text{lum}}{\sum_{s \in c} T \alpha}, \quad T = \prod_{\text{in front}} (1 - \alpha)$$

Everything is averaged over the six views. One call costs **14 ms** against 50–210 ms for a render, and the cached cubes are 34 MB for all 34 volumes.

3.2. Anatomy, not intensity bands

The classes started as Hounsfield bands (air / fat / soft / spongy / bone) and failed validation: “fat” (– 550 to – 30 HU) also collects lung edges and partial-volume voxels, “spongy” (170–600) collects contrast-filled vessels. Correlations against real renders were 0.46–0.68 for those two.

They are now anatomical, from the TotalSegmentator masks: **skeleton** (63 structures), **lungs** (5), **organs** (21), **muscle** (10), **vessels** (18). Everything else — subcutaneous fat, skin, bowel contents, the scanner table — is **other**: it occludes like anything else but is never a target.

Caveat: Only 7–20 % of body voxels carry any mask, so **other** is the majority of what occludes. And the four Slicer CTs have no masks at all; they fall back to intensity labels, and every result measured on them says so.

3.3. How the estimate was validated

The reference renders the same TF twice: once normally, once with one class’s **colour** set to black and its **opacity untouched**, using VTK’s label-map masking. The difference in mean luminance is exactly that class’s contribution, with occlusion unchanged.

Finding: Deleting a class’s voxels instead — the obvious first method — measures something else: the holes reveal what lies behind. Measured on ts_s0454 with a bone-dominant TF: 0.1481 normally, 0.1065 with the skeleton blacked out (its true contribution), 0.1194 with the skeleton deleted — **higher**, because the background showed through. Under the deletion method, thin surface tissue anti-correlated with its own visibility.

Transfer functions are sampled per class by sweeping that class’s own peak, since random TFs barely move a given class; coverage is checked separately on a global opacity sweep, because random TFs leave it near-constant.

Class	ts_s1379	ts_s1337	ts_s0454	Status
skeleton	1.000	0.999	1.000	validated
organs	0.995	0.994	0.969	validated
muscle	0.981	0.948	0.999	validated
vessels	0.992	0.989	(0.001 range)	validated on contrast scans
lungs	(0.0001 range)	(0.0000 range)	absent	below noise floor
coverage	1.000	1.000	1.000	rank agreement

Table 2: Pearson correlation between the estimate (visibility × brightness) and the label-masked render reference, 10 transfer functions per class. Classes whose reference contribution stays below the noise floor (0.002) are reported unvalidated rather than counted as passes.

Caveat: Lungs cannot be validated this way: lung tissue is nearly transparent, so its luminance contribution sits at the renderer’s noise floor. It is still a usable goal — the estimate tracks lung visibility — but the claim “validated against renders” does not extend to it.

4. The data

Split	Volumes	Source	Role
train	20	TotalSegmentator	policy training
validation	4	TotalSegmentator	checkpoint selection
test	6	TotalSegmentator	held-out result
out-of-source	4	3D Slicer CTs	different scanners, intensity labels

Table 3: Split by subject, stratified by body region (thorax, abdomen/pelvis, trunk, whole body). Five of the thirty are contrast angiographies. Selection is deterministic and recorded in `data/totalseg_manifest.json`.

All volumes load in canonical RAS orientation. For the Slicer NRRDs this is computed from each file’s own header rather than tuned by eye; the result was checked three independent ways, including that VTK preserves the file’s axis order.

5. Instructions and what counts as satisfying one

5.1. Goal classes

Goals target four classes: **skeleton**, **lungs**, **soft** (organs and muscle combined) and **vessels** (contrast scans only).

Finding: Organs and muscle cannot be separated by a transfer function — their intensities overlap almost entirely (medians 48 vs 44 HU on a contrast scan, – 3 vs 16 on a plain one). Vessels separate only with contrast (136 vs 48/44 HU). Asking for “more muscle, keep organs” or for vessels on a plain scan would be asking for something physically impossible, so the pipeline does not ask. Measurement keeps organs and muscle apart, so the thesis can report that they move together.

5.2. The goal vector and the distance

An instruction becomes a requested change per class: 16 values (requested visibility change d , mentioned-flag m , requested brightness change e , mentioned-flag n). Strength words map to $\log_{10}$ visibility changes (slightly 0.15, moderately 0.3, strongly 0.6); absolute levels (“high opacity skeleton”) map to a share of what that class can reach on **that** volume.

With $c = \log_{10}(\text{vis} + \epsilon) - \log_{10}(\text{vis}_{\text{start}} + \epsilon)$ and $b = \text{bright} - \text{bright}_{\text{start}}$:

$$D = \sum_c m|c - d| + \kappa \sum_c n|b - e| + \lambda \sum_c (1 - m) \max(0, |c| - \tau) + \lambda \kappa \sum_c (1 - n) \max(0, |b| - \tau)$$

with $\kappa = 1.5$, $\lambda = 0.3$, $\tau = 0.15$. **Attainment** is $1 - \frac{D_{\text{final}}}{D_{\text{start}}}$: 1 means the instruction was satisfied, 0 means nothing changed, negative means worse.

Finding: The tolerance τ is not cosmetic. Without it, moving one peak — which inevitably shifts what occludes what — is punished so hard that **doing nothing beats executing the instruction**. A reward where inaction wins is not a reward worth training against.

Caveat: Attainment is unbounded below: one bad episode on an easy goal measured -17.3 and swamped a mean over 16 episodes (mean -2.8 , median -0.59). Every aggregate in this document reports median, mean clipped to $[-1, 1]$, raw mean and share-positive.

6. Baselines

ID	Method	Median	Mean (clipped)	Share +
B4	hill-climb, 200 evaluations	+0.705	+0.652	100 %
B3	hill-climb, 10 evaluations	+0.288	+0.363	98 %
B0	do nothing	0.000	0.000	—
B1	today’s command executor	-0.021	-0.307	33 %
B2	random policy	-0.233	-0.167	37 %
B5	strip-everything heuristic	-0.635	-0.429	32 %

Table 4: 90 instructions across three volumes. B1 is what the existing rule-based system does with the same instruction.

Finding: B1 is good at brightness (+0.51 mean) and bad at everything involving visibility (-0.97 on relative instructions). That is the gap in one line: turning a colour knob maps directly to what you see, turning an opacity knob does not.

7. Two formulations

7.1. Ten-step refinement — the negative result

The first formulation gave the policy ten steps of ± 0.1 per parameter, rewarding the drop in goal distance at each step.

Setting	Steps	Median attainment	Verdict
20 volumes, all instruction kinds	20k $\rightarrow$ 120k	$-0.28 \rightarrow -0.13$	flat
1 volume, 1 instruction kind	20k $\rightarrow$ 100k	$-0.15 \rightarrow -0.98$	degrades
+ 4 gradient steps per env step	20k	-1.26	worse
+ lower exploration	20k	-0.42	worse
+ 10 $\times$ reward scale	20k	-0.08	no better

Table 5: The ten-step formulation does not learn, and gets worse with more training even when reduced to a single volume and a single instruction kind.

Finding: The diagnosis is structural, not a hyperparameter. A ten-step policy must commit to each parameter change blind, while the hill-climber it is measured against may try a change, measure it and reject it. The policy was being asked to learn an optimisation procedure — a harder problem than the mapping the thesis asks about, and one where a 10-evaluation search already scores +0.29.

Along the way the reward itself had to be repaired: start distances range from 0.15 (“brighten the skeleton slightly”) to 3.0 (“show only the lungs”), so raw distance drops made large-target episodes worth twenty times more than small ones. Normalising by D_{start} makes an episode’s undiscounted return equal its attainment.

7.2. One shot — what works

The policy reads the instruction, the volume’s intensity histogram, its current per-class visibility and brightness, the achievable ceiling per class, and the current parameters — 57 values — and outputs the transfer function directly. Reward is the attainment of that single output.

Steps	1 volume, 1 kind	20 volumes, all kinds	+ ceiling input
10k	—	− 0.059 (43 %)	− 0.031 (45 %)
20k	+0.381 (80 %)	− 0.029 (45 %)	+0.049 (60 %)
40k	+0.668 (96 %)	+0.099 (53 %)	—
70k	—	+0.266 (63 %)	—
120k	—	+0.140 (70 %)	+0.240 (80 %)
140k	—	+0.199 (70 %)	+0.322 (80 %)
150k	—	+0.180 (73 %)	+0.280 (85 %)

Table 6: Validation median attainment (share of episodes beating do-nothing in brackets). The same SAC that degraded in the ten-step formulation improves monotonically here.

Finding: Telling the policy what is achievable matters. `so_lo_max` per class — the most of that anatomy a single-peak transfer function can show on **this** volume — lifted the best validation median from +0.266 to +0.322 and reliability from 72 % to 85 %. Without it the policy has to infer each scan’s ceiling from the intensity histogram alone.

8. Held-out result

The policy’s best checkpoint, 200 fixed episodes on the six test subjects it never saw, every method scored on the same episodes:

Method	Evals	Median	Mean (clipped)	Share +	vs policy
B4 hill-climb	200	+0.730	+0.643	100 %	better, $p < 0.001$
policy + refinement	4	+0.316	+0.254	76 %	—
policy	0	+0.275	+0.205	73 %	—
B3 hill-climb	10	+0.263	+0.318	93 %	split by seed, see below
B0 do nothing	0	0.000	0.000	—	worse, $p < 0.001$
B1 today’s executor	0	− 0.022	− 0.172	37 %	worse, $p < 0.001$
B2 random	0	− 0.040	− 0.078	39 %	worse, $p < 0.001$
B5 strip-everything	0	− 0.191	− 0.290	30 %	worse, $p < 0.001$

Table 7: 200 fixed episodes on the six held-out test subjects, every method scored on the same episodes, median over three training seeds. Paired Wilcoxon signed-rank, two-sided. “Evals” counts visibility evaluations spent per instruction; the policy needs one to build its input features and none to search. Re-measured 2026-09-17 after an evaluation batch was found to have run on pre-fix scoring code; see the provenance note below.

What this supports, stated plainly: the learned policy **generalises to unseen patients** and beats doing nothing, today’s rule-based executor, a random policy and the hand-written heuristic, all at $p < 0.001$.

Finding: The policy reaches the 10-evaluation hill-climber’s median while spending no evaluations at all (+0.275 vs +0.263), and a learned proposal plus three refinements exceeds it at 4 evaluations (+0.316). Per seed the paired comparison splits — seed 0 favours search ($p = 3.0 \times 10^{-4}$), seeds 1 and 2 favour

the policy ($p = 0.054$, $p = 0.029$) – so the defensible claim is that the policy **matches** cheap search at a fraction of its cost, not that it beats it.

Caveat: Every figure in this table was wrong until 2026-09-17. The batch that produced the previously reported numbers started before the reachable-ceiling fix landed and wrote its files six hours after it, so it scored with the code it had imported rather than the code in the tree. Re-measuring moved the policy from +0.194 to +0.275 and every baseline with it, which is what identifies the fault as the ruler rather than the method. `provenance.py` now records the git commit and a fingerprint of the imported scoring modules in every result file, and `rl.vis_eval --show` refuses to print one silently when that fingerprint no longer matches.

Caveat: Reliability is the remaining gap and it is not small: the hill-climber improves 93 % of episodes, the policy 73 % and the hybrid 76 %, and the hill-climber’s clipped mean (+0.318) is still the better figure. The fair claim is that the policy matches cheap search **in the middle of the distribution** while being cheaper, not that it is better overall. Its worst episodes remain worse.

9. Where reinforcement learning actually earns its place

The argument for a learned policy is not efficiency against a formula. It is that **search becomes impossible when the objective is a person**. A hill-climber needs 10–200 evaluations per instruction; a human cannot be asked 200 times what they think of a rendering. Even against a learned reward model each evaluation costs a render plus an embedding (0.3–0.5 s), so a 200-evaluation search is roughly 100 seconds per spoken command – unusable in an interactive viewer, let alone in VR.

That is the setting the preference pipeline targets, and why tier 1 is scaffolding: it establishes that the objective is learnable and supplies the baselines, the environment and the trained proposal distribution that the preference work reuses.

10. Collecting human preferences

Each item: a scan, a random start transfer function, an instruction sampled from the grammar (restricted to classes that scan contains), and two candidates – half the items pit two stochastic samples of the policy against each other, half pit the policy against B1, B3, B5 or a random perturbation. Pairs that differ by less than 0.1 in $\log_{10}$ visibility and 0.05 in brightness for every class are discarded. Each candidate is rendered as the six standard views, so the rater judges exactly what the reward model will later receive.

Roughly one item in ten repeats an earlier one with the sides swapped. That measures the rater’s self-consistency, which is the ceiling any reward model can reach – a number worth reporting alongside the model’s accuracy.

Every judgment stores the instruction, the start, both parameter vectors, both sources, the measured features, which candidate the **objective** preferred, the choice and the decision time, in one file that training reads directly.

11. Limitations

- Lungs are a goal but not validated against renders (contribution below the render’s noise floor).
- Organs and muscle are one target, because no transfer function separates them.
- Vessels are only a goal on contrast scans (5 of 30 volumes).
- The out-of-source volumes use intensity labels, not anatomy.
- The policy matches cheap search on the median but not on reliability (67 % vs 90 % of episodes improved); its worst episodes are worse.
- Peak positions are fixed; only height, width and brightness are learned. Lung goals are reachable only because a peak is seeded at – 800 HU.

- One seed evaluated so far; two more are training.

12. Reproducing the numbers

```
# data: 30 CT volumes, splits, anatomical labels
python -m tools.select_totalseg
python -m tools.build_visibility_cache

# the measurement and its validation gate
python -m tools.validate_visibility ts_s1379 ts_s1337 ts_s0454

# baselines
python -m tools.baseline_report --volumes ts_s1379 ts_s1337 ts_s0454 --
instructions 30

# the policy
python -m rl.oneshot_train --timesteps 150000 --seed 0 --out out/rl_v2/
oneshot_v2_seed0
python -m rl.vis_eval --policy out/rl_v2/oneshot_v2_seed0/best.zip --split test
\
    --episodes 200 --formulation one_shot --refine 3

# preference collection
python server.py          # then open /collect
```

Design documents: docs/superpowers/specs/2026-09-15-rl-v2-visibility-rlhf-design.md and the plans in docs/superpowers/plans/.